

DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 3:

Parallelism within single processor

Locality and sources of parallelism in simulations

Parallelism and Locality in Simulation

- Parallelism and data locality both critical to performance
 - Recall that moving data is the most expensive operation
- Real world problems have parallelism and locality
 - Many objects operate independently of others.
 - Objects often depend much more on nearby than distant objects.
- Scientific models may introduce more parallelism
 - When a continuous problem is discretized, time dependencies are generally limited to adjacent time steps.
- Many problems exhibit parallelism at multiple levels.

Common ideas / issues

- Load balancing
 - Imbalance can be caused by poor parallelism/distribution
 - Can be static or dynamic
- Locality
 - Want big blocks with low surface-to-volume ratio
 - Minimizes communication
 - Can generalize ideas to graph partitioning
- Tensions and tradeoffs
 - Irregular spatial decompositions for load balance at the cost of complexity, and perhaps extra overhead
 - Example: particle-mesh methods — can't manage moving particles and fixed meshes simultaneously without communicating

Particle Systems

- A particle system has
 - a finite number of particles
 - moving in space according to Newton's Laws (i.e. $F = ma$)
 - time is continuous
- Examples:
 - stars in space with laws of gravity
 - electron beam in semiconductor manufacturing
 - atoms in a molecule with electrostatic forces
 - neutrons in a fission reactor

Forces on each particle

- External force: externally imposed electric field in electron beam
 - Parallelism is the simplest; locality is not an issue
- Nearby force: Van der Waals forces in fluid ($1/r^6$)
 - Requires interaction and therefore communication; usual parallel model is domain decomposition
 - Load imbalance, if particles cluster
- Far-field force: gravity, electrostatics
 - Requires all-to-all interaction and therefore communication; simplest algorithm is $O(n^2)$

Sparsity

Assume solving ODE $x'(t) = f(x) = A * x(t)$, where A is a sparse matrix

$$A = \begin{pmatrix} a_{11} & a_{12} & 0 & 0 & \dots \\ a_{21} & a_{22} & a_{23} & 0 & \\ 0 & a_{32} & a_{33} & a_{34} & \\ \vdots & & \ddots & \ddots & \ddots \end{pmatrix}$$

for example representing a system of ordinary differential equations (ODEs).

Sparse because each component (f_j) only depends on a few others (x_i) - physical locality.

Types of analysis

Consider $x' = Ax - b$. Might want:

- Static analysis ($Ax - b = 0$)
 - Boils down to $Ax = b$ (e.g. for Newton-like steps)
 - Can solve directly or iteratively
 - Sparsity matters a lot!
- Dynamic analysis (compute $x(t)$ for many values of t)
 - Involves time stepping (explicit or implicit)
 - Implicit methods involve linear/nonlinear solves
 - Need to understand stiffness and stability issues
- Modal analysis (e.g. compute eigenvalues of A)

Explicit/Implicit time stepping

Explicit method:

- Example: forward Euler; $x[i + 1] = x[i] + dt * A * x[i]$
- Next step depends only on earlier steps
- Simple algorithms; sparse matrix vector multiply
- May have stability/stiffness issues which needs to take very small time steps

Implicit method:

- Example: backward Euler, $(I - dt * A) * x[i + 1] = x[i]$
- Next step depends on itself and on earlier steps
- Algorithms involve solving a sparse linear system of equations — complication, communication!
- Larger time steps, each step costs more

A common kernel

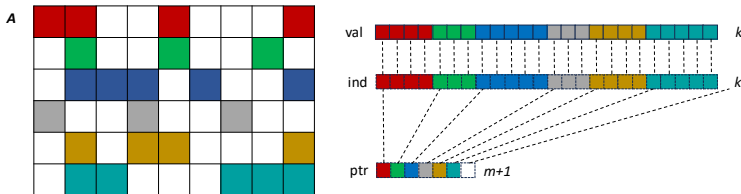
All these problems reduce to sparse matrix problems

- Explicit time marching: sparse matrix-vector multiplication
- Implicit time marching: solve a sparse linear system
- Iterative/Direct linear solvers
- Iterative eigensolvers

We do not want to neither store nor compute on these 0's.

We need to figure out how to do these fast!

Example: Compressed sparse row storage

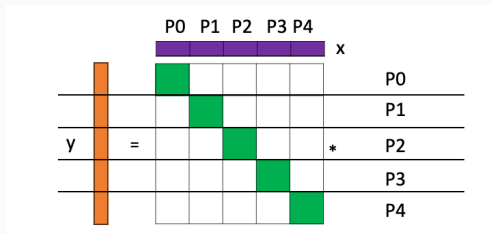


```
1  \\Matrix-vector multiply kernel:  $y(i) = y(i) + A(i, j) * x(j)$ 
2
3  for each row  $i$ 
4      for  $k = \text{ptr}[i] \text{ to } \text{ptr}[i+1]-1$ 
5           $y[i] = y[i] + \text{val}[k] * x[\text{ind}[k]]$ 
```

This can be even more compact by organizing by blocks.

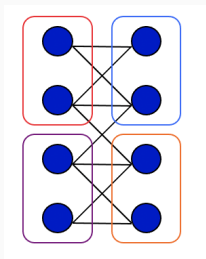
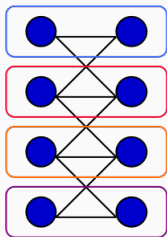
Example: Parallel Sparse Matrix-vector multiplication

- Block diagonal matrix structure is ideal for parallelism
 - p (number of processors) blocks, can all be computed locally.
 - If no non-zeros outside these blocks, no communication needed.
 - If non-zeros outside these blocks, try to reorder so most nonzeros put in diagonal blocks, few outside.



$y = y + A * x$, only store, do arithmetic, on nonzero entries

Sparsity and partitioning



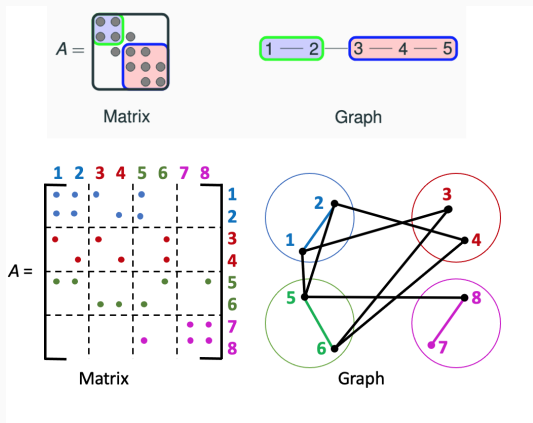
Graph partitioning assigns subgraphs to processors:

- Subgraphs are same size (load balance): evenly distribute subgraphs to nodes
- Cut size is minimal (minimize communication): minimize edge crossings

Easy for meshes, NP-hard in general, so we will approximate.

Graph Partitioning and Sparse Matrices

- Relationship between matrix and graph



If divided over 3 procs, there are 14 nonzeros outside diagonal blocks, which represent 7 (bidirectional) edges.

1-D Heat Equation

Heat Equation (Conservation of Energy and Fourier's Law):

$$\rho c \frac{\partial u}{\partial t} = k \frac{\partial^2 u}{\partial x^2},$$

where $k/(\rho c)$ is the **diffusivity** considered constant here.

Consider the following boundary and initial conditions, for

$0 < x < L$:

$$u(0, t) = u_0 \quad u(L, t) = u_L,$$

known as **Dirichlet** boundary conditions (we can also specify a heat flux, Neumann boundary conditions, or combination thereof, known as mixed or Robin boundary conditions), and

$$u(x, 0) = f(x) \quad 0 < x < L.$$

Fourier Series Solution to the 1-D Heat Equation

A solution to the homogeneous Dirichlet boundary conditions problem is

$$u(x, t) = \sum_{n=1}^{\infty} a_n \sin\left(\frac{n\pi x}{L}\right) e^{-\left(\frac{n\pi}{L}\right)^2 \left(\frac{k}{\rho c}\right) t}$$

where

$$f(x) = \sum_{n=1}^{\infty} a_n \sin\left(\frac{n\pi x}{L}\right)$$

Finite difference numerical methods for 1-D heat equation



Explicit Method $O(\Delta t, \Delta x^2)$

$$u_i^{n+1} = \left(\frac{k\Delta t}{\rho c \Delta x^2} \right) (u_{i+1}^n + u_{i-1}^n) + \left(1 - \frac{2k\Delta t}{\rho c \Delta x^2} \right) u_i^n$$

with $\frac{2k\Delta t}{\rho c \Delta x^2} \leq 1$.

Implicit Method $O(\Delta t, \Delta x^2)$

$$u_i^n = - \left(\frac{k\Delta t}{\rho c \Delta x^2} \right) (u_{i+1}^{n+1} + u_{i-1}^{n+1}) + \left(1 + \frac{2k\Delta t}{\rho c \Delta x^2} \right) u_i^{n+1}$$

Finite difference numerical methods for 1-D heat equation



Explicit Method $O(\Delta t, \Delta x^2)$

$$u_i^{n+1} = \left(\frac{k\Delta t}{\rho c \Delta x^2} \right) (u_{i+1}^n + u_{i-1}^n) + \left(1 - \frac{2k\Delta t}{\rho c \Delta x^2} \right) u_i^n$$

with $z = \frac{k\Delta t}{\rho c \Delta x^2}$, and $u_i^n = u[i, j]$ we have:

$$u[i, j+1] = z * u[i-1, j] + (1 - 2z)u[i, j] + z * u[i+1, j]$$

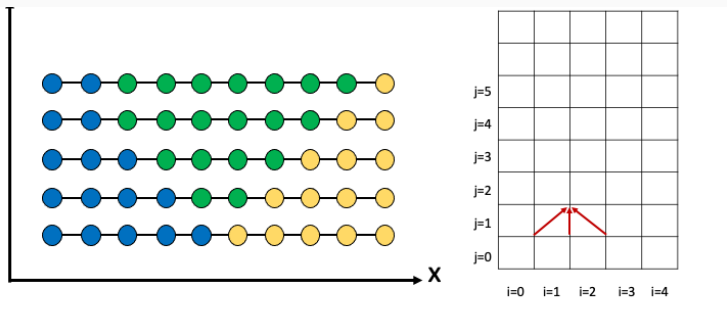
for $i=1$ to $N-1$

$$u[i, j+1] = z * u[i-1, j] + (1 - 2 * z) * u[i, j] + z * u[i+1, j]$$

This corresponds to matrix-vector-multiply

Explicit Solution of the Heat Equation

$$u[i, j + 1] = z * u[i - 1, j] + (1 - 2z)u[i, j] + z * u[i + 1, j]$$



Nearest neighbor interactions per step $\Rightarrow$
finite rate of numerical information propagation

Spatial discretization

Heat equation with $u(0) = u(1) = 0$

$$u[i, j + 1] = z * u[i - 1, j] + (1 - 2z)u[i, j] + z * u[i + 1, j]$$

This corresponds to $u[: j + 1] = Au[: j]$ where A (tridiagonal) is:

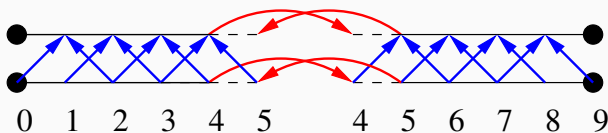
$$A = \begin{bmatrix} 1 - 2z & z & & & \\ z & 1 - 2z & z & & \\ & \ddots & \ddots & \ddots & \\ & & z & 1 - 2z & z \\ & & & z & 1 - 2z \end{bmatrix}$$



Graph and 3 point stencil

Explicit time stepping in parallel

Sparse matrix vector multiply, via Graph Partitioning



```
for t = 1 to N
  communicate boundary data ("ghost cell")
  take time steps locally
end
```

Generalizes to multiple dimensions.

Problem with explicit approach for heat (parabolic): numerical instability; solution blows up eventually if $z > 0.5$.

Implicit time stepping

- Backward Euler uses backward difference for time derivative:

$$-z * u_{i+1}^{n+1} + (1 + 2z)u_i^{n+1} - z * u_{i-1}^{n+1} = u_i^n$$

This corresponds to $Au[j+1] = u[j]$ where A (tridiagonal) is:

$$A = \begin{bmatrix} 1 + 2z & -z & & & \\ -z & 1 + 2z & -z & & \\ & \ddots & \ddots & \ddots & \\ & & -z & 1 + 2z & -z \\ & & & -z & 1 + 2z \end{bmatrix}$$

- Taking a time step $\equiv$ sparse matvec with A^{-1}
- No time step restriction for stability
- But each step involves finding A^{-1} ; good if you have a fast linear solver.

Explicit and implicit

Explicit:

- Propagates information at finite rate
- Steps look like sparse matvec (in linear case)
- Stable step determined by fastest time scale

Implicit:

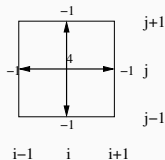
- No need to resolve fastest time scales
- Steps can be long... but expensive
 - Linear/nonlinear solves at each step
 - Often these solves involve sparse matrix-vector multiplication

Consider 2D Poisson

$$-\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f$$

- Prototypical elliptic problem (steady state)
- Similar to a backward Euler step on heat equation

Poisson problem discretization



$$u_{i,j} = h^{-2} (4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1})$$

$$A = \begin{bmatrix} 4 & -1 & & & & \\ -1 & 4 & -1 & & & \\ & -1 & 4 & & & \\ -1 & & & 4 & -1 & \\ & -1 & & -1 & 4 & -1 \\ & & & & -1 & 4 \end{bmatrix}$$

Poisson solvers in 2D/3D

N total unknowns; which solver would you select?

Method	Time(3D)	Memory(3D)	PRAM
Dense LU	N^3	N^2	N
Band LU	N^2 ($N^{7/3}$)	$N^{3/2}$ ($N^{5/3}$)	N
Jacobi	N^2 ($N^{5/3}$)	N	$N(N^{2/3})$
Explicit inv	N^2	N^2	$\log N$
Conjugate Gradients	$N^{3/2}(N^{4/3})$	N	$N^{1/2(1/3)} \log N$
Red-black SOR	$N^{3/2}$ ($N^{4/3}$)	N	$N^{1/2(4/3)}$
Sparse LU	$N^{3/2}$ (N^2)	$N \log N$ ($N^{4/3}$)	$N^{1/2(2/3)}$
FFT	$N \log N$	N	$\log N$
Multigrid	N	N	$\log^2 N$

PRAM: Idealized parallel model with zero cost communication

Ref: Demmel, *Applied Numerical Linear Algebra*, SIAM, 1997.

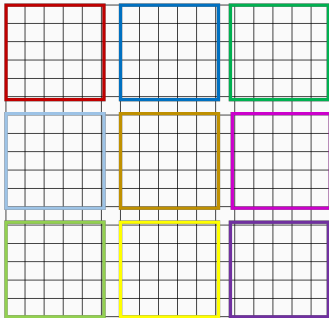
Decision tree to help choose algorithms: www.netlib.org/linalg/

General implicit picture

- Implicit solves or steady state $\implies$ solving systems
- Nonlinear solvers generally linearize
- Linear solvers can be
 - Direct (hard to scale)
 - Iterative (often problem-specific)
- Iterative solves boil down to matrix-vector multiplication; graph partitioning

Parallelism in Regular meshes

- Computing a Stencil on a regular mesh
 - Need to communicate mesh points near boundary to neighboring processors.
 - Often done with ghost regions
 - Surface-to-volume ratio keeps communication down, but
 - Still may be problematic in practice
 - Implemented using “ghost” regions; adds memory overhead



PDE solver summary

- Can be implicit or explicit (as with ODEs)
 - Explicit (sparse matrix-vector multiplication) — fast, but small timesteps
 - works fine for hyperbolic PDEs
 - Implicit (sparse solve)
 - Direct solvers are hard!
 - Sparse solvers turn into matrix-vector multiplication
- Differential operators turn into local mesh stencils
 - Matrix connectivity looks like mesh connectivity
 - Can partition into subdomains that communicate only through boundary data
 - More on graph partitioning later
- Not all nearest neighbor ops are equally efficient!
 - Depends on mesh structure
 - Also depends on flops/point